FTP WEB Manager

A web-based FTP/FTPS management platform for managing FTP users, FTP services, file access, and related administration through a browser-based interface.

GitHub Repository:
https://github.com/SudarshanMakur-21/FTP-WEB-Manage

---

1. Overview

FTP WEB Manager consists of:

- React + TypeScript frontend
- FastAPI backend
- PostgreSQL database
- Redis
- vsftpd FTP/FTPS server
- Nginx reverse proxy/web server
- Gunicorn + Uvicorn
- systemd service management
- SELinux
- firewalld
- Let's Encrypt SSL/TLS support

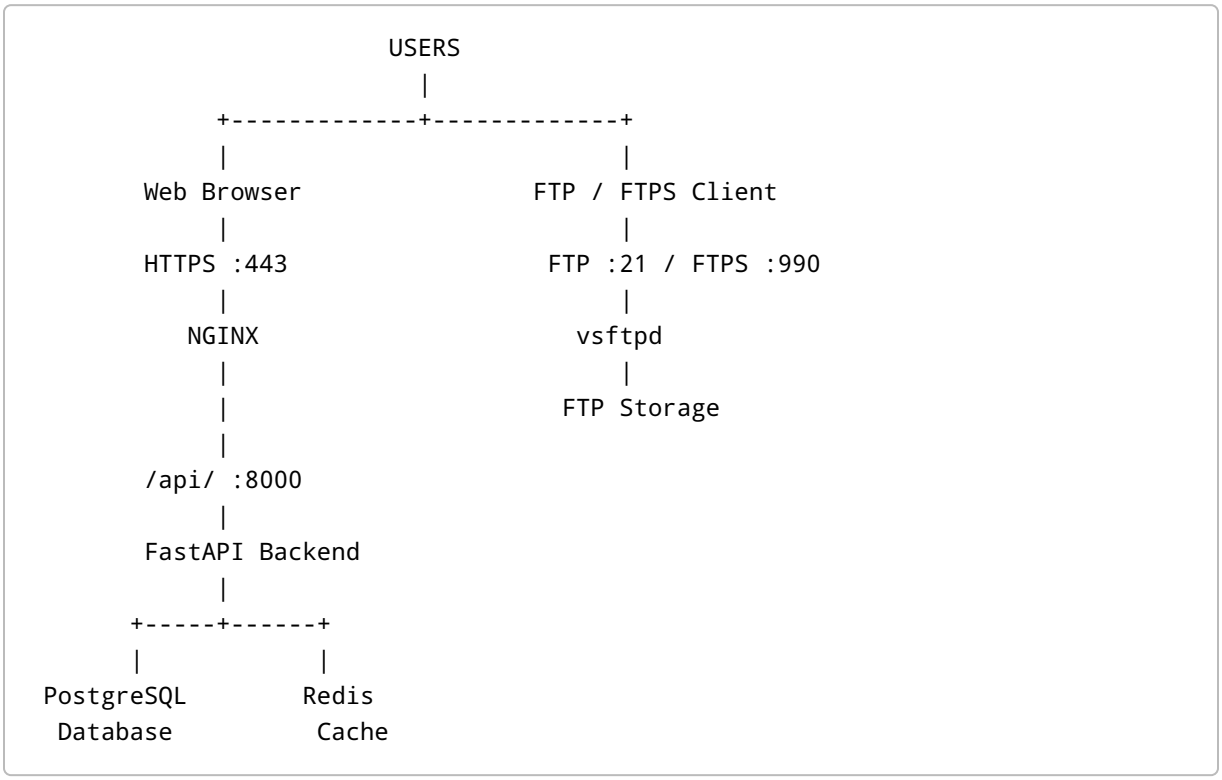
The current repository includes a dedicated RHEL deployment directory:

```
deployment/
└── rhel/
    ├── scripts/
    │   ├── install.sh
    │   ├── setup-ssl.sh
    │   └── update.sh
    ├── systemd/
    │   ├── ftp-manager-backend.service
    │   └── ftp-manager-frontend.service
    └── ...
```

The project is specifically designed with RHEL 9.6 deployment in mind. The installer itself identifies RHEL 9.6 as its target platform.

---

2. Architecture



Application flow

```
Browser
   |
   | HTTPS
   v
Nginx
   |
   | /api/
   v
FastAPI
   |
   +----> PostgreSQL
   |
   +----> Redis
   |
   +----> vsftpd configuration / FTP management
```

The backend systemd service currently runs Gunicorn with four Uvicorn workers on TCP port `8000`.

---

3. Recommended Deployment Platform

Primary platform

```
RHEL 9.6
```

Other RHEL-compatible distributions

The deployment can potentially be adapted to:

```
Rocky Linux 9
AlmaLinux 9
```

However, the supplied installer explicitly targets RHEL and enables RHEL repositories where possible.

For production, RHEL 9.6 is the recommended deployment platform.

---

4. Recommended Server Specification

For a small/medium deployment:

Component	Recommended
OS	RHEL 9.6
CPU	4 vCPU
RAM	8 GB
OS Disk	80 GB SSD
FTP Storage	Based on requirement
Network	1 Gbps preferred
IP	Static IP
DNS	FQDN recommended
SELinux	Enforcing
Firewall	firewalld

For high concurrent FTP usage or large file transfers, increase CPU, RAM, storage performance, and network capacity accordingly.

5. Required Network Ports

Port	Protocol	Purpose
80	TCP	HTTP
443	TCP	HTTPS
21	TCP	FTP
990	TCP	FTPS
40000-50000	TCP	FTP Passive Mode

Do **not** expose these services publicly unless required:

```
5432/tcp  PostgreSQL
6379/tcp  Redis
8000/tcp  FastAPI
```

The backend should normally listen locally behind Nginx.

---

6. Prerequisites

Before deployment, prepare:

- RHEL 9.6 server
- Root or sudo access
- Valid RHEL subscription
- Static IP
- DNS record
- Internet connectivity
- Required firewall access
- FTP storage
- Domain name for HTTPS/FTPS

Example:

```
ftp.example.com
        |
        +----> RHEL Server IP
```

---

7. Verify RHEL

```
cat /etc/redhat-release
```

Expected:

```
Red Hat Enterprise Linux release 9.6
```

Check architecture:

```
uname -m
```

Check SELinux:

```
getenforce
```

Recommended:

```
Enforcing
```

---

8. Clone the Repository

The repository is currently public, so HTTPS cloning can be used.

```
sudo -i

cd /opt

git clone https://github.com/SudarshanMakur-21/FTP-WEB-Manage.git ftp-manager

cd /opt/ftp-manager
```

Verify:

```
ls -la
```

Expected:

```
backend/
frontend/
deployment/
.gitignore
.gitattributes
```

The current repository contains these major directories.

---

9. Application Directory Structure

Recommended production layout:

```
/opt/ftp-manager/
|
+-- backend/
|
+-- frontend/
|
+-- deployment/
|   +-- rhel/
|
+-- venv/
|
+-- .env
|
+-- logs/
|
+-- ssl/
```

Additional system configuration:

```
/etc/nginx/conf.d/ftp-manager.conf

/etc/vsftpd/vsftpd.conf

/etc/vsftpd/virtual_users.db

/etc/systemd/system/ftp-manager-backend.service

/etc/systemd/system/ftp-manager-frontend.service
```

```
/var/www/ftp-manager/
```

---

10. Application Components

Backend

The backend uses FastAPI and includes:

```
FastAPI
Uvicorn
SQLAlchemy
Alembic
PostgreSQL asyncpg
Redis
Celery
Paramiko
aioftp
python-ldap
Gunicorn
```

These dependencies are pinned in:

```
backend/requirements.txt
```

The current repository uses FastAPI `0.109.0`, SQLAlchemy `2.0.25`, asyncpg `0.29.0`, Redis `5.0.1`, Celery `5.3.6`, and Gunicorn `21.2.0`, among others.

Frontend

The frontend is a React/TypeScript application built using Vite.

The production build command is:

```
pnpm run build
```

The current `package.json` defines:

```
React
TypeScript
Vite
React Router
Axios
TanStack React Query
Tailwind CSS
```

---

11. Automated RHEL Installation

The repository provides:

```
deployment/rhel/scripts/install.sh
```

The script installs/configures:

```
Python 3.11
Node.js 20
npm
pnpm
PostgreSQL
Redis
Nginx
vsftpd
Certbot
Git
GCC
OpenSSL development libraries
firewalld
SELinux utilities
logrotate
```

This is implemented directly in the current installer.

Run installer

From the repository:

```
cd /opt/ftp-manager/deployment/rhel/scripts
```

```
chmod +x *.sh

bash install.sh
```

The installer must be run as root.

---

12. IMPORTANT: Production Warning

The current installer contains default credentials.

The current script creates:

```
PostgreSQL database:
    ftpmanager

PostgreSQL user:
    ftpuser

PostgreSQL password:
    ftppass
```

It also creates:

```
Admin:
    admin@ftp.local

Password:
    admin123
```

These values are present in the current `install.sh`.

DO NOT use these credentials in production.

Before production deployment:

- Generate a strong PostgreSQL password.
- Generate a strong application secret.
- Change the administrator password.
- Restrict CORS.
- Configure the real domain.
- Configure proper SSL certificates.
- Review vsftpd permissions.

- Review SELinux permissions.

---

13. PostgreSQL

The installer initializes PostgreSQL if required and enables the service.

Manual verification:

```
systemctl status postgresql
```

Database:

```
ftpmanager
```

Check:

```
sudo -u postgres psql -c "\l"
```

Check user:

```
sudo -u postgres psql -c "\du"
```

---

14. Redis

Enable Redis:

```
systemctl enable --now redis
```

Check:

```
systemctl status redis
```

Test:

```
redis-cli ping
```

Expected:

```
PONG
```

The current installer also enables Redis persistence using AOF.

---

15. Python Environment

The installer creates:

```
/opt/ftp-manager/venv
```

Verify:

```
/opt/ftp-manager/venv/bin/python --version
```

Verify pip:

```
/opt/ftp-manager/venv/bin/pip --version
```

Check application dependencies:

```
/opt/ftp-manager/venv/bin/pip list
```

---

16. Environment File

The application configuration is stored at:

```
/opt/ftp-manager/.env
```

The current installer creates this file automatically.

Example production configuration:

```
DATABASE_URL=postgresql+asyncpg://ftpuser:STRONG_DB_PASSWORD@127.0.0.1:5432/
ftpmanager

SECRET_KEY=GENERATED_RANDOM_SECRET

ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=10080

FTP_HOST=127.0.0.1
FTP_PORT=21
FTP_TLS_PORT=990
FTP_PASSIVE_PORTS=40000:50000

VSFTPD_CONFIG_PATH=/etc/vsftpd/vsftpd.conf
VSFTPD_USER_DB_PATH=/etc/vsftpd/virtual_users.db
VSFTPD_USER_CONFIG_DIR=/etc/vsftpd/user_config

VSFTPD_SSL_CERT=/etc/ssl/ftpmanager/fullchain.pem
VSFTPD_SSL_KEY=/etc/ssl/ftpmanager/privkey.pem

SSL_CERT_DIR=/etc/ssl/ftpmanager

REDIS_URL=redis://127.0.0.1:6379/0

BACKEND_CORS_ORIGINS=["https://ftp.example.com"]
```

Generate the application secret:

```
openssl rand -hex 32
```

Protect the file:

```
chown ftpmanager:ftpmanager /opt/ftp-manager/.env
chmod 600 /opt/ftp-manager/.env
```

---

17. Frontend Build

Frontend directory:

```
cd /opt/ftp-manager/frontend
```

Install dependencies:

```
pnpm install
```

Build:

```
pnpm run build
```

The repository's frontend build script is:

```
tsc && vite build
```

The generated application is then served by Nginx.

---

18. Nginx

The current installer creates:

```
/etc/nginx/conf.d/ftp-manager.conf
```

The application architecture uses:

```
Browser
   |
Nginx :80/:443
   |
FastAPI :8000
```

The current configuration proxies:

```
/api/
```

to:

```
127.0.0.1:8000
```

and serves the frontend from:

```
/var/www/ftp-manager
```

Check Nginx:

```
nginx -t
```

Enable:

```
systemctl enable --now nginx
```

---

19. Backend systemd Service

The repository contains:

```
deployment/rhel/systemd/ftp-manager-backend.service
```

The service:

- Runs as `ftpmanager`
- Uses `/opt/ftp-manager/backend`
- Loads `/opt/ftp-manager/.env`
- Runs Gunicorn
- Uses Uvicorn workers
- Listens on `0.0.0.0:8000`
- Automatically restarts
- Uses systemd security restrictions

Install:

```
cp
/opt/ftp-manager/deployment/rhel/systemd/ftp-manager-backend.service
/etc/systemd/system/
```

Then:

```
systemctl daemon-reload

systemctl enable --now ftp-manager-backend
```

Check:

```
systemctl status ftp-manager-backend
```

Check port:

```
ss -lntp | grep 8000
```

---

20. Frontend systemd Service

The repository includes:

```
deployment/rhel/systemd/ftp-manager-frontend.service
```

This is a lightweight systemd placeholder for the Nginx-served frontend. It does not run a Node/Vite development server; its `ExecStart` is `/bin/true`, while Nginx serves the actual built frontend.

Therefore:

```
Frontend runtime:
    Nginx

Not:
    npm start
    vite dev
```

---

21. vsftpd

The application uses:

```
vsftpd
```

The installer creates:

```
/etc/vsftpd/user_config/
/etc/vsftpd/virtual_users.txt
/etc/vsftpd/virtual_users.db
```

and configures PAM for virtual users.

Check:

```
systemctl status vsftpd
```

Enable:

```
systemctl enable vsftpd
```

---

22. FTP Passive Mode

The configured passive range is:

```
40000-50000/tcp
```

This range must be allowed through:

```
RHEL firewalld
Network firewall
NAT/firewall
Cloud security group
```

if applicable.

---

23. Firewall

The installer opens:

```
HTTP
HTTPS
FTP
990/tcp
40000-50000/tcp
```

Check:

```
firewall-cmd --list-all
```

Recommended explicit configuration:

```
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service=https
firewall-cmd --permanent --add-port=21/tcp
firewall-cmd --permanent --add-port=990/tcp
firewall-cmd --permanent --add-port=40000-50000/tcp

firewall-cmd --reload
```

Verify:

```
firewall-cmd --list-ports
```

---

24. SELinux

Do not disable SELinux.

Check:

```
getenforce
```

Expected:

```
Enforcing
```

The current deployment enables:

```
setsebool -P httpd_can_network_connect on
```

for Nginx-to-backend communication.

It also configures the frontend web root with the appropriate SELinux context.

Check:

```
ls -Zd /var/www/ftp-manager
```

---

25. FTPS / SSL

The repository provides:

```
deployment/rhel/scripts/setup-ssl.sh
```

Usage:

```
cd /opt/ftp-manager/deployment/rhel/scripts

bash setup-ssl.sh ftp.example.com admin@example.com
```

The script:

1. Installs Certbot if required.
2. Obtains a Let's Encrypt certificate.
3. Copies certificates to `/etc/ssl/ftpmanager`.
4. Configures vsftpd SSL.
5. Configures Nginx HTTPS.
6. Redirects HTTP to HTTPS.
7. Creates certificate renewal automation.

---

26. DNS Requirement for SSL

Before running SSL setup:

```
ftp.example.com
        |
        v
RHEL SERVER PUBLIC IP
```

Verify DNS:

```
dig +short ftp.example.com
```

or:

```
nslookup ftp.example.com
```

The result must point to the correct server.

---

27. HTTPS Architecture

After SSL configuration:

```
HTTP :80
   |
   +----> HTTPS :443
             |
           Nginx
             |
        FastAPI :8000
```

The current SSL script configures TLS 1.2 and TLS 1.3 for Nginx.

---

28. Initial Web Access

After installation:

```
http://SERVER-IP
```

After HTTPS:

```
https://ftp.example.com
```

The current installer creates the initial administrator:

```
Email:
admin@ftp.local

Password:
admin123
```

IMPORTANT

Immediately change this password.

For production, replace the default account initialization mechanism with a secure first-login/password provisioning process.

---

29. Service Status

Check all application services:

```
systemctl status ftp-manager-backend
systemctl status ftp-manager-frontend
systemctl status nginx
systemctl status postgresql
systemctl status redis
systemctl status vsftpd
```

Quick check:

```
systemctl --type=service --state=running |
egrep 'ftp-manager|nginx|postgresql|redis|vsftpd'
```

---

30. Application Logs

Backend

```
journalctl -u ftp-manager-backend -f
```

Last 100 lines:

```
journalctl -u ftp-manager-backend -n 100 --no-pager
```

Nginx

```
tail -f /var/log/nginx/error.log
```

```
tail -f /var/log/nginx/access.log
```

vsftpd

```
tail -f /var/log/vsftpd.log
```

SELinux

```
ausearch -m AVC -ts recent
```

---

31. Health Checks

Backend

```
curl http://127.0.0.1:8000
```

Nginx

```
curl -I http://127.0.0.1
```

HTTPS

```
curl -I https://ftp.example.com
```

PostgreSQL

```
sudo -u postgres psql -c "SELECT version();"
```

Redis

```
redis-cli ping
```

Expected:

```
PONG
```

---

32. Common Problems

Backend does not start

Run:

```
systemctl status ftp-manager-backend
```

Then:

```
journalctl -u ftp-manager-backend -n 100 --no-pager
```

Check:

```
cat /opt/ftp-manager/.env
```

Do not expose the `.env` contents publicly.

Nginx configuration error

Run:

```
nginx -t
```

Then:

```
journalctl -u nginx -n 100 --no-pager
```

PostgreSQL connection failure

Check:

```
systemctl status postgresql
```

Then:

```
sudo -u postgres psql
```

Verify:

```
\l
\du
```

Redis failure

```
systemctl status redis
```

Then:

```
redis-cli ping
```

FTP connection failure

Check:

```
systemctl status vsftpd
```

Check:

```
ss -lntp | grep ':21'
```

Check firewall:

```
firewall-cmd --list-all
```

FTP passive mode failure

Verify:

```
40000-50000/tcp
```

is allowed through every firewall/NAT between the FTP client and server.

SELinux denial

Check:

```
ausearch -m AVC -ts recent
```

Do not immediately disable SELinux.

---

33. Updating the Application

The repository provides:

```
deployment/rhel/scripts/update.sh
```

Usage:

```
cd /opt/ftp-manager/deployment/rhel/scripts

bash update.sh
```

The current update script performs repository/application update operations and service-related deployment tasks.

Before updating production:

```
1. Backup PostgreSQL
2. Backup .env
3. Backup Nginx configuration
4. Backup vsftpd configuration
5. Test application update
6. Run database migrations
7. Restart backend
8. Rebuild frontend
9. Verify web access
10. Verify FTP/FTPS
```

---

34. PostgreSQL Backup

Create backup directory:

```
mkdir -p /backup/ftp-manager
```

Backup:

```
sudo -u postgres pg_dump ftpmanager
> /backup/ftp-manager/ftpmanager-$(date +%Y%m%d-%H%M%S).sql
```

Verify:

```
ls -lh /backup/ftp-manager/
```

---

35. Configuration Backup

Backup important configuration:

```
tar -czf
/backup/ftp-manager/config-$(date +%Y%m%d-%H%M%S).tar.gz
/opt/ftp-manager/.env
/etc/nginx/conf.d/ftp-manager.conf
/etc/vsftpd/vsftpd.conf
/etc/vsftpd/user_config
```

Protect backups because they may contain sensitive configuration.

---

36. Production Security Checklist

Before production go-live:

- [ ] RHEL fully patched
- [ ] SELinux enabled
- [ ] firewalld enabled
- [ ] Strong PostgreSQL password
- [ ] Strong application `SECRET_KEY`
- [ ] Default admin password changed
- [ ] HTTPS enabled
- [ ] FTPS enabled where required
- [ ] DNS configured
- [ ] CORS restricted to production domain
- [ ] PostgreSQL not exposed publicly
- [ ] Redis not exposed publicly
- [ ] Backend port 8000 not exposed publicly
- [ ] FTP passive range restricted as appropriate
- [ ] FTP storage permissions reviewed

- [ ] SSH access restricted
- [ ] Backup configured
- [ ] Log rotation configured
- [ ] Monitoring configured
- [ ] Certificate renewal tested
- [ ] Recovery procedure tested

---

37. Current Installer Security Notes

The supplied `install.sh` is useful for initial deployment but should be hardened before production.

Current items requiring review:

Default database password

```
ftpuser / ftppass
```

Default administrator

```
admin@ftp.local / admin123
```

Example domain

```
ftp.example.com
```

Example email

```
admin@example.com
```

Broad FTP SELinux permission

The installer currently enables:

```
setsebool -P allow_ftpd_full_access on
```

This should be reviewed against the actual FTP storage/access requirements rather than enabled automatically in a hardened environment.

---

38. Recommended Production Improvements

Before production release, the deployment scripts should ideally be changed to:

```
1. Generate PostgreSQL password automatically
2. Generate admin password or require it interactively
3. Never print credentials in installation output
4. Require DOMAIN as an installation parameter
5. Require ADMIN_EMAIL
6. Require ADMIN_PASSWORD
7. Generate SECRET_KEY automatically
8. Remove localhost:3000 from production CORS
9. Configure vsftpd explicitly
10. Avoid unnecessary SELinux broad permissions
11. Validate passive FTP range
12. Add pre-install backup
13. Add rollback support
14. Add deployment health checks
15. Add systemd hardening
16. Add automated certificate renewal validation
17. Add application database backup
```

---

39. Recommended Production Deployment

For production:

```
                  Internet
                     |
               Firewall / NAT
                     |
          +----------+----------+
          |                     |
     HTTPS :443             FTPS :990
          |                     |
       Nginx                 vsftpd
          |                     |
          |                FTP Storage
          |
     FastAPI :8000
          |
     +----+----+
```

```
       |         |
 PostgreSQL    Redis
```

Keep the following services private:

```
PostgreSQL :5432
Redis :6379
FastAPI :8000
```

Only expose:

```
80
443
21
990
40000-50000
```

as required by the deployment.

---

40. Quick Installation

For a **test/non-production** RHEL 9.6 server:

```
sudo -i

cd /opt

git clone https://github.com/SudarshanMakur-21/FTP-WEB-Manage.git ftp-manager

cd /opt/ftp-manager/deployment/rhel/scripts

chmod +x *.sh

bash install.sh
```

Then:

```
systemctl status ftp-manager-backend
systemctl status nginx
systemctl status postgresql
```

```
systemctl status redis
systemctl status vsftpd
```

Open:

```
http://SERVER-IP
```

For production, complete the security configuration before exposing the application to the Internet.

---

41. SSL Installation

After DNS is correctly configured:

```
cd /opt/ftp-manager/deployment/rhel/scripts

bash setup-ssl.sh ftp.example.com admin@example.com
```

Then verify:

```
nginx -t

systemctl status nginx

systemctl status vsftpd
```

Test:

```
curl -I https://ftp.example.com
```

The repository's SSL script configures both Nginx HTTPS and vsftpd SSL and installs a renewal cron job.

---

42. File Locations

Component	Location
Application	`/opt/ftp-manager`
Backend	`/opt/ftp-manager/backend`

Component	Location
Frontend	`/opt/ftp-manager/frontend`
Python venv	`/opt/ftp-manager/venv`
Environment	`/opt/ftp-manager/.env`
Frontend web root	`/var/www/ftp-manager`
Nginx config	`/etc/nginx/conf.d/ftp-manager.conf`
vsftpd config	`/etc/vsftpd/vsftpd.conf`
FTP virtual DB	`/etc/vsftpd/virtual_users.db`
FTP user configuration	`/etc/vsftpd/user_config`
SSL files	`/etc/ssl/ftpmanager`
Backend systemd	`/etc/systemd/system/ftp-manager-backend.service`
Logs	`/var/log/ftp-manager`

---

43. Useful Commands

Restart backend

```
systemctl restart ftp-manager-backend
```

Restart Nginx

```
systemctl restart nginx
```

Restart PostgreSQL

```
systemctl restart postgresql
```

Restart Redis

```
systemctl restart redis
```

Restart FTP

```
systemctl restart vsftpd
```

Check ports

```
ss -lntp
```

Check firewall

```
firewall-cmd --list-all
```

Check SELinux

```
getenforce
```

Check disk

```
df -h
```

Check memory

```
free -h
```

Check CPU

```
uptime
```

---

44. Production Go-Live Sequence

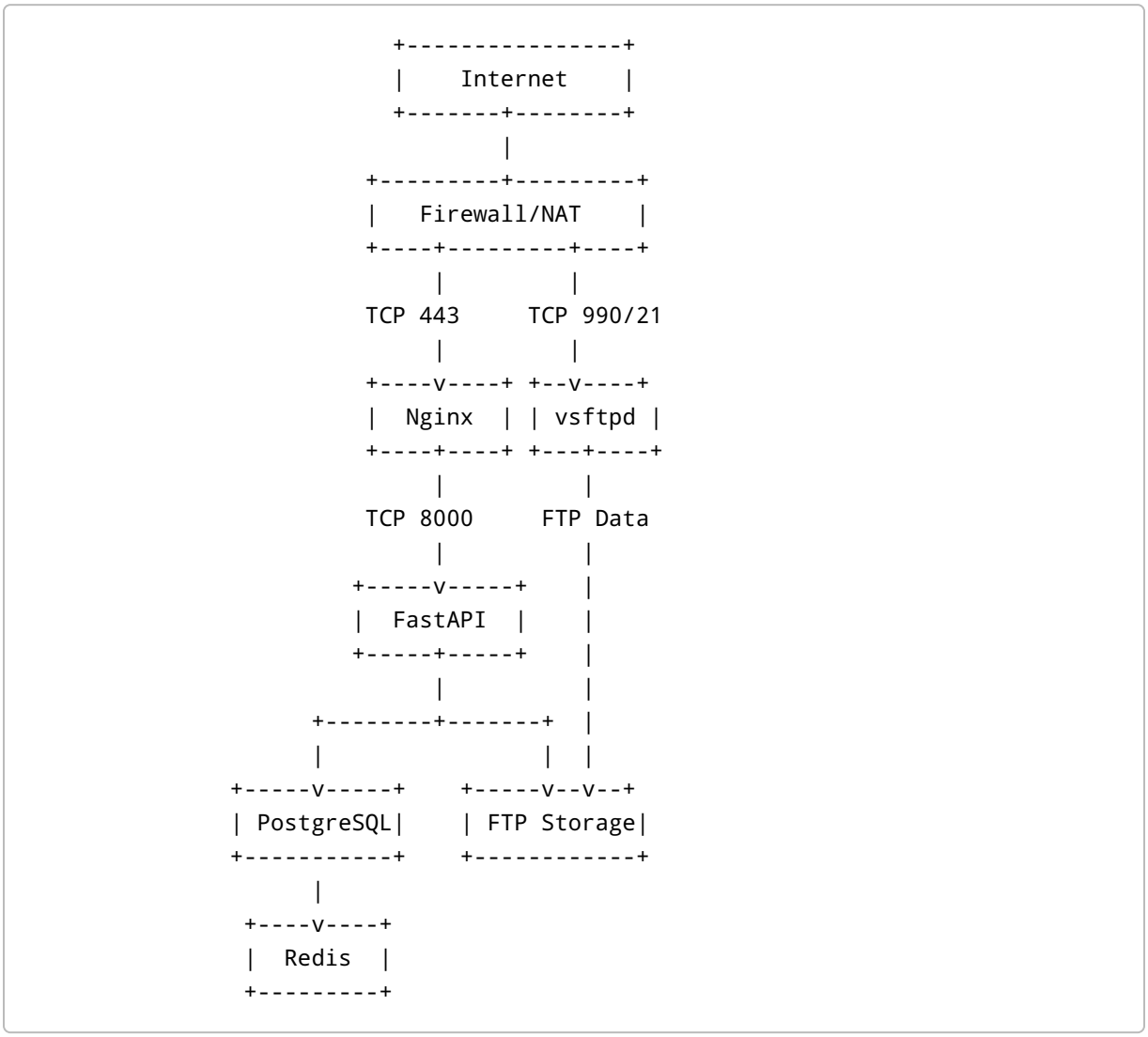
Use this order for a new production server:

```
1. Install RHEL 9.6
        |
2. Patch RHEL
        |
```

```
3. Configure static IP/DNS
        |
4. Configure subscription/repositories
        |
5. Clone repository
        |
6. Review deployment scripts
        |
7. Install dependencies
        |
8. Configure PostgreSQL
        |
9. Configure Redis
        |
10. Configure application environment
        |
11. Install Python dependencies
        |
12. Build frontend
        |
13. Configure FastAPI/systemd
        |
14. Configure Nginx
        |
15. Configure vsftpd
        |
16. Configure SELinux
        |
17. Configure firewalld
        |
18. Test HTTP
        |
19. Configure HTTPS
        |
20. Configure FTPS
        |
21. Change default administrator password
        |
22. Configure backups
        |
23. Configure monitoring
        |
24. Production go-live
```

---

45. Final Architecture



License / Project Information

Refer to the repository for the current source code, project license, and latest deployment changes:

https://github.com/SudarshanMakur-21/FTP-WEB-Manage